面向个人长期记录的

智能体记忆系统《忆》

课程项目说明文档

记忆准入 · 版本治理 · 分层存储 · 证据检索

课程名称	设计大数据基础
项目名称	面向个人长期记录的智能体记忆系统《忆》
姓名	成可心
学号	3240102321
完成日期	2026 年 11 月 3 日

摘要

《忆》是一个面向个人长期记录场景的智能体记忆系统。项目关注的不是“保存更多聊天记录”，而是信息进入记忆库之后如何被筛选、更新、淡忘和重新检索。系统将普通问答、个人记忆查询和基于记忆的综合建议分开处理；将用户事实、附件资料和第三方内容分开存储；将当前状态、历史版本和长期结论分层管理。配置文字模型 API 后，模型参与语义判断、通用问答和个性化建议；配置视觉模型 API 后，可进一步完成图片场景理解，但不能直接写入记忆，也不能绕过本地证据和版本规则。

项目已完成可交互原型，支持文字、语音、图片、多个文件与文字组合发送，支持记忆历书、记忆库、取代链、淡忘恢复、资料库、流程审计和数据导出。主实验使用 219 条模拟个人记录和 48 道问题，本方案综合准确率为 100.0%，高于“全量历史直接作为上下文”的 79.2%；自动测试 157/157 通过。



图 1 产品界面

一、项目目标

1.1 面向场景与使用者

项目面向需要长期记录个人信息的使用者，例如持续记录饮食偏好、学习安排、健康提醒、出行计划、消费习惯和阶段性目标的人。记录来源可能是临时输入、语音转写、图片、文件或连续对话，内容会随时间变化，并不始终保持一致。

普通对话模型能够利用当前上下文回答问题，但在长期使用中容易出现以下问题：

1. **新旧事实混用**：同一事项多次修改后，旧版本仍可能被当作当前事实。
2. **记忆无限累积**：所有内容长期保留会增加检索噪声，直接删除又会失去历史依据。
3. **写入边界不清**：问题、建议请求、附件正文和第三方信息可能被误写为用户事实。
4. **查询范围不清**："现在""曾经""最近""怎么变化的"需要不同证据，不能用同一种检索方式处理。
5. **结果难以验证**：如果每次都把全部历史交给大模型，证据选择和冲突裁决难以复现。

1.2 项目目标

项目将目标确定为四项：

- 建立可控的记忆准入机制，区分明确记录、可自动记录的高置信事实、待确认候选、普通对话和附件资料。
- 建立可追溯的版本治理机制，保存当前状态、历史版本、修改顺序和取代关系。
- 建立面向个人小语料的检索机制，根据问题选择当前事实、历史状态、时间线或稳定画像。
- 建立可复现的验证体系，通过统一数据、题集、消融实验和自动测试说明各机制的实际作用。

二、实现方法

2.1 数据、工具与技术栈

项目实验数据由 219 条模拟个人记录组成，覆盖饮食、健康、学习、计划、出行、财务等主题；主测试集包含 48 道问题，覆盖近期事实、矛盾处理、总结归纳、时间回溯、时间线和诱导拒答六类场景。另设置固定 76 题和扩展 126 题语义压力集、3 组拟真笔记回放以及 40 用例端到端评测。模拟数据避免使用真实敏感个人信息，便于重复运行和公开展示。

主要工具与技术如下：

类别	使用内容
后端与接口	Python 3.11、FastAPI、Uvicorn
前端	HTML、CSS、JavaScript 单页界面
检索与语义	jieba、BM25、时间窗、主题图、RRF 排序融合
数据与本体	JSON/JSONL、RDFLib、RDFS/OWL 导出
文档处理	pypdf、python-docx、openpyxl、python-pptx、Pillow
多模态	浏览器语音接口、Windows OCR、可选视觉模型接口
模型服务	OpenAI 兼容接口，当前可配置 DeepSeek 等文本或视觉模型
测试与制图	unittest、Matplotlib、固定评测脚本
AI 编程工具	ChatGPT / Codex，用于需求拆解、代码实现、故障定位和文档整理

AI 编程工具不直接决定实验结论。需求先被转换为可观察行为和边界条件，再通过固定数据、统一题集和自动测试验证。实验脚本固定运行参数和评分条件，开放式回答质量另通过实际交互检查。

2.2 总体架构

系统采用“统一接入、统一编排、任务分流、共享状态”的结构。文字、语音、图片和文件先被解析为带来源标签的统一请求，再由编排器识别任务类型、执行记忆准入，并选择仅检索、检索后推理或通用问答。写入任务依次完成记录抽取和冲突治理；查询任务依次完成三路召回、RRF 融合、证据范围裁决和回答审核。两类任务共享同一分层记忆与资料状态，检索只能读取受版本约束的证据，模型不能绕过准入和版本机制直接读写长期记忆。底部验证流程统一保存关键事件，并通过固定题集、消融实验和自动回归检查机制。

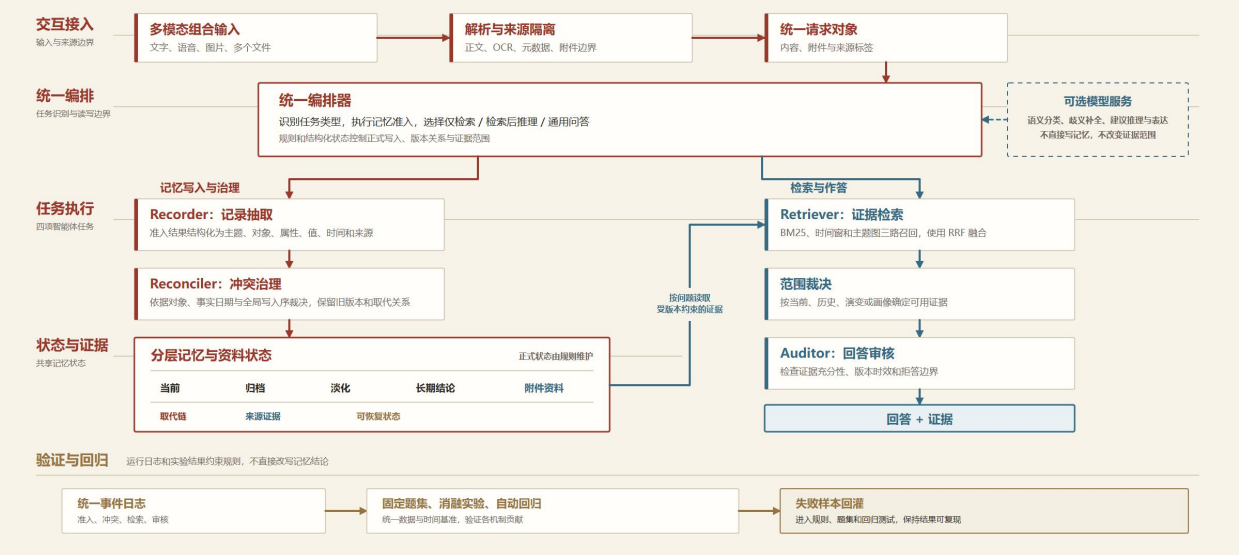


图 2 系统总体架构

四项智能体任务分别是记录抽取、冲突治理、证据检索和回答审核，由统一编排器共享记忆状态和日志，并非四个相互独立的模型服务。规则与结构化状态负责准入结果、版本关系和证据范围；配置文字模型 API 后，模型参与语义分类与回答生成，负责意图分类、歧义补全、建议推理、综合表达和通用问答，但不能直接写入正式记忆或改变证据范围。

2.3 记忆准入与安全边界

系统不会把所有用户输入自动写入记忆，而是将输入划分为五类：

输入类型	处理方式
明确记录	用户明确要求“记住”“记录”，通过安全检查后写入
高置信个人事实	明确的长期偏好、计划或已发生事件，可自动记录
待确认候选	可能长期有用但稳定性不足，等待用户保留或忽略
普通对话	问题、建议请求、闲聊和临时表达，不直接写入
附件资料	文档正文、OCR 和第三方内容进入资料库，不自动归属于用户

这一机制用于处理两类容易混淆的情况。例如，“我不吃香菜”属于稳定偏好，可以成为长期记忆；“我今天想吃麻辣烫，你说选什么口味”首先是建议请求，应先检索已有偏好并回答。如果后续用户明确选定某种口味，系统可记录当天事件；只有在多次出现或用户明确确认时，才形成长期偏好。

写入前还会执行来源和敏感信息检查。附件中的文字、网页提示或文档指令只作为资料内容，不作为系统指令执行；模型生成的推断不能直接进入正式记忆，候选内容必须能够在用户原文中找到依据。

2.4 结构化语义与版本治理

每条记忆保存主题、对象、属性、取值、动作、日期、重要度、来源和全局写入序。查询也转换为相同的语义框架：

```
topic / entities / facets / values / scope / excluded_entities
```

结构化语义用于区分“同主题不同对象”和“同对象不同属性”。例如，咖啡的烘焙偏好不能被错误用于麻辣烫建议；问题中的假设也不能反向污染证据，“我不是喜欢特辣吗”中的“特辣”只作为待核验内容，而不是检索必须满足的事实。

当新旧记录指向同一对象且状态冲突时，系统根据事实日期和全局写入序判断先后。日期相同时仍能识别后一次修改，避免只依赖时间戳造成顺序不稳定。旧版本不会被直接覆盖，而是保留取代关系、修改时间和来源，供历史查询与演变查询使用。

2.5 分层记忆与主动淡忘

系统内部状态包括：

状态	含义
active	当前有效或近期高活跃记录
compressed	低活跃但仍保留依据的归档记录
superseded	已被新版本取代的历史记录
discarded	退出日常检索、但允许恢复的淡忘记录
conclusion	同主题长期演变形成的稳定模式或结论

报告与界面中所称“归档层”由 `compressed` 和 `superseded` 共同组成。主动淡忘表示记录退出默认检索范围，不等于物理删除；需要回顾历史时仍可恢复或进入档案检索。

2.6 查询路由与证据检索

系统首先判断问题是否依赖个人记忆，再判断是否需要 LLM 参与综合，由此形成三种模式：

模式	适用问题	执行方式
`retrieval_only`	当前事实、日期、历史状态、明确配置	本地检索和状态裁决，确定性回答
`retrieval_llm`	基于个人信息的建议、比较、总结	先检索个人证据，再由 LLM 组织答案
`general_llm`	普通知识、写作、与个人记录无关的问题	不读取个人记忆

检索由三条通道组成：

- **BM25**：处理精确词项、名称和具体事实。
- **时间窗**：将“上周”“国庆”“十月以来”等表达映射为日期区间。
- **主题图**：扩展同主题事件、长期结论和前后变化。
- **RRF 融合**：合并不同通道的排序，降低单一检索方式失效的影响。

检索结果还需要按问题范围裁决：

- `current`：只输出当前有效状态。
- `history`：定位曾经、最初或指定日期的状态。
- `evolution`：保留前后变化和取代链。
- `profile`：汇总稳定偏好、习惯和长期模式。

“我最近做了什么”属于宽泛时间回顾，默认展开最近七天的多条记录；如果当前七天没有记录，系统回退到记忆库最近有记录的同长度窗口，并明确标注实际日期范围。“最近一次跑步”和“现在还在跑步吗”则分别属于最后一次事件查询和当前状态查询。

2.7 多模态与文档处理

对话入口支持文字与多个图片、多个文件组合发送。图片可使用系统 OCR 提取文字；配置视觉模型 API 后，可进一步分析物体、场景和图文关系。文件支持 TXT、MD、CSV、JSON、LOG、PDF、DOCX、XLSX 和 PPTX，扫描 PDF 可渲染后进行 OCR。旧版 `.doc/.xls/.ppt` 需先转换为现代格式。

附件以文件卡片显示，正文默认折叠，点击后查看解析内容。仅发送附件时，系统自动补充“请分析这些附件”。附件分析与个人记忆写入相互独立：资料正文进入资料库，只有用户明确要求记录或后续确认的个人事实才进入记忆库。

2.8 Vibe Coding 实现过程

项目采用自然语言需求驱动、快速实现、持续验证的 Vibe Coding 方式，但没有将开发过程停留在一次性生成代码。主要流程如下：

1. **用真实交互描述需求**：先描述用户会怎样输入、系统应怎样回答、哪些内容不能写入记忆。
2. **将现象归纳为机制问题**：例如附件与文字不能组合发送，不按单个文件补丁处理，而是统一建立附件队列、消息模型和资料/记忆边界。

3. **由 AI 编程工具辅助实现**：使用 ChatGPT / Codex 分析代码结构、生成局部实现、定位接口和前端状态问题。
4. **建立通用规则与测试**：将“是否需要个人记忆”“是否需要 LLM”“是否允许写入”分别建模，避免针对某一句话增加特例。
5. **回归测试与实验复跑**：每次修改后运行单元测试、语义压力集、主实验和端到端用例，检查是否破坏原有机制。
6. **同步更新文档与 PPT**：实验数字、架构术语和产品能力统一从最新代码与结果文件校对，避免展示材料与实现不一致。

开发中暴露出的主要问题包括：纯规则对语义边界覆盖不足、模型参与后的回答路径不清、附件被误当作记忆、近期查询被压缩为单条状态、文件卡片无法折叠等。最终修改均落实到路由、准入、共享语义或附件模型层，并通过跨场景测试验证，而不是为单个提问增加固定回答。

三、最终成果

3.1 已完成功能

模块	已实现内容
对话入口	普通问答、个人查询、基于记忆的建议、明确记录
多模态输入	文字、语音、图片、多文件与文字组合发送
语音交互	浏览器语音转写、回答朗读
记忆管理	活跃、归档、淡忘、恢复、长期结论
版本治理	冲突检测、当前状态裁决、取代链和历史回溯
检索回答	BM25、时间窗、主题图、RRF、证据范围裁决
资料管理	附件解析、独立资料库、折叠查看、删除
可视化界面	总览、记忆历书、记忆库、待确认、流程、洞察
审计与导出	处理事件、检索证据、JSON 导出、RDF 本体导出

3.2 实际运行效果

系统能够在同一入口中区分三类行为：

- 对“我现在每月定投多少”执行本地事实检索，只返回当前有效值。
- 对“按我的口味推荐一种麻辣烫口味”先检索个人偏好，再由模型给出建议。
- 对“麻辣烫一般有哪些口味”执行通用问答，不读取个人记忆。

对于同一事项的连续变化，系统分别支持当前查询、历史查询和演变查询，并可在界面中查看取代链。对于图片和文件，系统能够联合分析，但不会把附件中的人物、事件或说明静默写成用户经历。

四、验证与改进

4.1 主实验

主实验比较三种架构：

- A：只使用最近 3 条记录。
- B：保存全部 219 条历史，但不进行准入、分层和冲突治理。
- C：使用本项目的准入、分层、版本治理和三路检索。

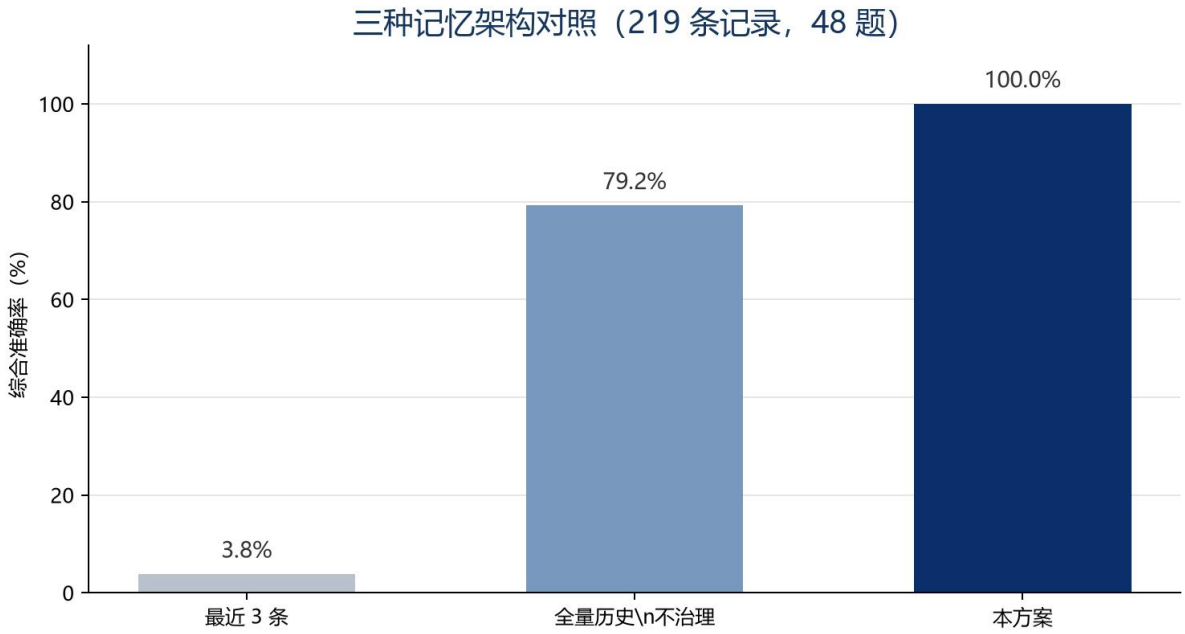


图 3 三架构主实验

架构	近期	矛盾	总结	时间	时间线	诱导拒答	综合
A 最近 3 条	0%	0%	17%	0%	11%	0%	3.8%
B 全量历史、不治理	96%	90%	61%	100%	33%	0%	79.2%
C 本方案	**100%**	**100%**	**100%**	**100%**	**100%**	**100%**	**100.0%**

本方案比全量历史方案提高 20.8 个百分点。219 条记录最终形成 49 条活跃记忆、142 条归档记录和 28 条淡忘记录，说明效果提升并非来自把全部内容长期放入上下文，而是来自准入、版本治理和按范围检索。

4.2 消融实验

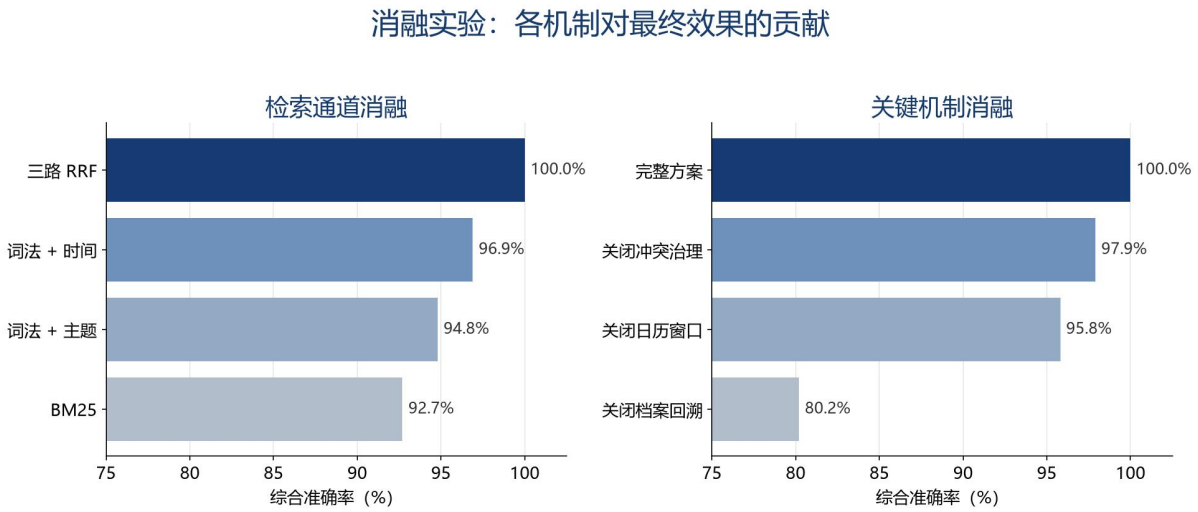


图 4 消融实验

检索消融中，BM25 单独使用为 92.7%，加入时间窗后为 96.9%，加入主题图后为 94.8%，三路 RRF 达到 100.0%。关闭日历窗口后降至 95.8%，关闭档案回溯后降至 80.2%，说明时间语义和历史档案是当前题集中的主要贡献来源。

关闭独立冲突治理后综合准确率为 97.9%，完整任务分工为 100.0%，并形成 12/12 组对象级冲突账本和取代链。该结果说明冲突治理的价值不仅体现在最终分数，也体现在版本关系和审计记录的完整性。

4.3 工程验证

验证项目	结果
固定语义压力集	76/76
扩展语义压力集	126/126
拟真笔记回放	3/3
40 用例端到端评测	98.8%
自动测试	157/157
冲突发现与解决	12/12
RDF 本体导出	1233 条三元组

实验使用固定数据、时间基准、检索配置和评分条件，保证不同方案可以重复比较。开放式建议、语义判断和附件理解另通过端到端用例与实际交互检查。

4.4 当前不足

1. 主数据仍以模拟记录为主，需要使用经过脱敏的长期真实数据验证持续使用效果。
2. 当前确定性评测适合比较机制，但不能完整评价开放式回答的语言质量。
3. 系统按单用户本地使用设计，多用户部署还需要账号、权限、数据加密和隔离。

4. 本地 OCR 依赖 Windows 能力；跨平台公网部署需改用 Tesseract、云 OCR 或视觉模型。
5. 当前存储以本地 JSON/JSONL 为主，大规模数据和并发访问需要数据库与向量索引支持。

4.5 后续改进

- 建立脱敏真实数据集，增加跨月、跨主题和长期偏好漂移测试。
- 为模型参与的综合回答增加延迟监控、缓存和降级策略，缩短建议生成的等待时间。
- 引入用户可配置的自动记忆阈值和敏感主题白名单，进一步提高写入可控性。
- 将本地存储迁移到支持权限和持久化的数据库，完善多用户隔离。
- 为 Linux 部署补充跨平台 OCR，并增加文件大小、解析超时和恶意文件检查。

五、代码与展示方式

5.1 代码仓库

代码仓库已公开发布，可直接访问：

```
https://github.com/isnlc/Yi
```

仓库包含项目源代码、`requirements.txt`、README、技术文档、测试和运行说明，不包含 `.env`、API 密钥、缓存文件和个人运行数据。

5.2 本地运行

下载或克隆代码后，进入包含 requirements.txt 和 .env.example 的仓库根目录，执行以下命令：

```
Copy-Item .env.example .env

# 在 .env 中填写文字模型接口地址、密钥和模型名称
# 并保持 MEMAGENT_LLM=1
pip install -r requirements.txt
python web/app.py --port 8765
```

浏览器访问：

```
http://127.0.0.1:8765
```

系统会自动读取仓库根目录下的 .env。文字模型接口用于语义分类、通用问答、个性化建议、开放式总结和附件内容综合。接口暂时不可用时，系统仍保留记忆治理、确定性事实检索、文档解析和 Windows OCR；图片中的物体、场景和图文关系分析需要单独配置视觉模型接口。

5.3 课堂展示与视频

本项目不提供在线 Demo。课堂汇报采用“PPT 讲解 + 约 2 分钟展示视频”的形式，必要时可在本机补充操作。

视频依次展示新记录写入与版本治理、当前事实检索与取代链追溯、基于个人记忆的综合建议，以及文字、图片和多个文件组合发送与资料隔离。同时展示回答朗读、悬浮入口、记忆历书、记忆库、处理流程和洞察页面。

六、结论

《忆》将长期记忆问题拆分为准入、治理、检索和审核四项任务，重点解决个人记录中的版本变化、错误写入、检索范围和结果可追溯问题。实验表明，单纯保存全部历史并不能替代结构化治理；完整方案在统一题集上达到 100.0%，并通过压力测试、回放测试和自动回归验证。项目已经形成可运行、可演示、可复现的课程原型，后续工作主要集中在真实长期数据、多用户安全和公网部署。